Indian Institute of Information Technology Design and Manufacturing Kurnool

Department of Computer Science & Engineering

GVR Compiler

Complete Compiler Implementation
with Automated Testing Framework

Project Report

Submitted by:

GAUTAM V

Roll Number: 524CS0017

Department of Computer Science & Engineering

524cs0017@iiitk.ac.in

Academic Year 2025-2026

March 2026

Contents

1 Executive Summary

GVR is a complete compiler that translates high-level source code into x86 32-bit assembly language. The project includes a comprehensive automated testing framework that validates compiler output through compilation, assembly, linking, and execution verification. This document provides complete documentation for building, using, and testing the compiler.

2 Overview

2.1 Key Features

- Complete compilation pipeline from source to executable
- Target architecture: x86 32-bit (IA-32)
- AT&T syntax assembly generation
- Automated end-to-end testing framework
- Comprehensive error reporting and diagnostics
- Compile-time error detection (division by zero, type errors)
- Makefile-based build automation
- Support for interactive programs with input/output
- N-dimensional array support

2.2 Project Structure

Component	Description
`src/`	Compiler source code files
`include/`	Header files and declarations
`gvr.out`	Compiled compiler executable
`Makefile`	Build configuration and automation
`run_tests.py`	Automated test runner script
`tests/`	Test suite with source, expected output, and input files

3 Project File Structure

This section provides a detailed explanation of each file and directory in the GVR Compiler project, describing their purpose and functionality.

3.1 Root Directory Files

File/Directory	Description
`gvr.out`	Main compiler executable. This is the compiled binary that translates source code files into x86-32 assembly. Generated by running `make`.
`Makefile`	Build automation script. Defines compilation rules, dependencies, and commands for building the compiler. Supports targets: `make` (build), `make clean` (remove artifacts), and `make rebuild` (clean + build).
`run_tests.py`	Automated test runner script written in Python. Orchestrates the complete testing pipeline: compiles test programs, assembles them, links executables, runs with input, validates output against expected results, and generates test reports.
`README.md`	Project overview and quick-start guide. Contains basic usage instructions, build requirements, and links to detailed documentation.

3.2 Source Code Directory (`src/`)

Core Implementation: Contains all compiler implementation files written in C/C++. Each file implements a specific phase of the compilation pipeline.

File	Description
`main.c`	**Entry Point:** Orchestrates the entire compilation pipeline. Parses command-line arguments, initializes data structures, and sequentially triggers the lexer, parser, analyzer, and code generator.
`io.c`	**File Management:** Handles system-level input/output for the compiler itself. Responsible for reading `.gv` source files into memory buffers and writing the final generated assembly to `.s` output files.

File	**Description** (Continued)
`lexer.c`	**Lexical Analyzer:** Scans the raw source code character-by-character and converts it into a continuous stream of tokens. Identifies keywords, operators, literals, and identifiers while filtering out whitespace and comments.
`token.c`	**Token Definitions:** Defines the underlying data structures, enumerations, and utility functions for tokens. Acts as the standard data format passed between the lexer and the parser.
`parser.c`	**Syntax Analyzer:** Consumes the token stream and applies language grammar rules to construct an Abstract Syntax Tree (AST). Handles syntax error detection and context-aware reporting.
`AST.c`	**Abstract Syntax Tree:** Defines the node structures for all syntactic elements (expressions, statements, declarations). Provides utilities for node creation, tree linking, and memory management.
`list.c`	**Data Structures:** Implements dynamic lists or dynamic array utilities used internally by the compiler to manage token streams, variable-length AST children, and intermediate code sequences.
`visitor.c`	**AST Traversal:** Implements the Visitor pattern to cleanly traverse the AST. Used to separate the logic of compilation passes (like semantic checking or intermediate code generation) from the tree structure itself.
`types.c`	**Type System:** Manages semantic type definitions, type sizes (e.g., 4-byte integers), and type-checking logic to ensure operations and function calls are mathematically and logically valid.
`tac.c`	**Intermediate Representation:** Lowers the high-level AST into Three-Address Code (TAC). Breaks down complex nested expressions into simple, sequential instructions to bridge the gap between syntax and hardware assembly.
`as_frontend.c`	**Assembly Generator:** Translates TAC instructions into raw x86-32 AT&T assembly. Handles register allocation, stack frame setup, memory offsets, system call injections, and defines the `_start` entry point.
`builtins.c`	**Standard Library:** Stores and injects the core assembly routines required by the language environment, including dynamic heap allocators, I/O functions (`print`, `input`), and runtime startup definitions.

3.3 Header Files Directory (`include/`)

Interface Definitions: Header files declare function prototypes, data structures, constants, and type definitions used across compilation stages.

File	Description
`as_frontend.h`	Assembly generation interface. Declares functions for translating TAC instructions into x86-32 AT&T assembly and managing registers.
`ast.h`	AST node definitions. Declares structures for expressions, statements, and declarations, along with node creation utilities.
`builtins.h`	Standard library interface. Declares standard assembly routines like heap allocation, core I/O functions, and startup definitions.
`io.h`	File I/O interface. Declares functions for reading source files into memory buffers and writing assembly output to files.
`lexer.h`	Lexical analyzer interface. Declares lexer state structures and the core functions needed to scan source code and generate tokens.
`list.h`	Dynamic data structures. Declares dynamic array/list structures and memory management utilities used throughout the compiler.
`macros.h`	Compiler macros. Defines global utility macros for error handling, debugging, and common operations used across the codebase.
`parser.h`	Syntax analyzer interface. Declares parsing functions for language grammar rules and AST construction utilities.
`tac.h`	Intermediate Representation. Declares structures for Three-Address Code (TAC) instructions, operands, and generation functions.
`token.h`	Token definitions. Defines token type enumerations and the core structure representing lexical units (type, value, line, column).
`types.h`	Type system interface. Defines internal language types, memory sizes, and declarations for semantic type-checking functions.
`visitor.h`	AST Traversal interface. Declares the visitor pattern structures and traversal functions used for various compilation passes.

3.4 Test Directory (`tests/`)

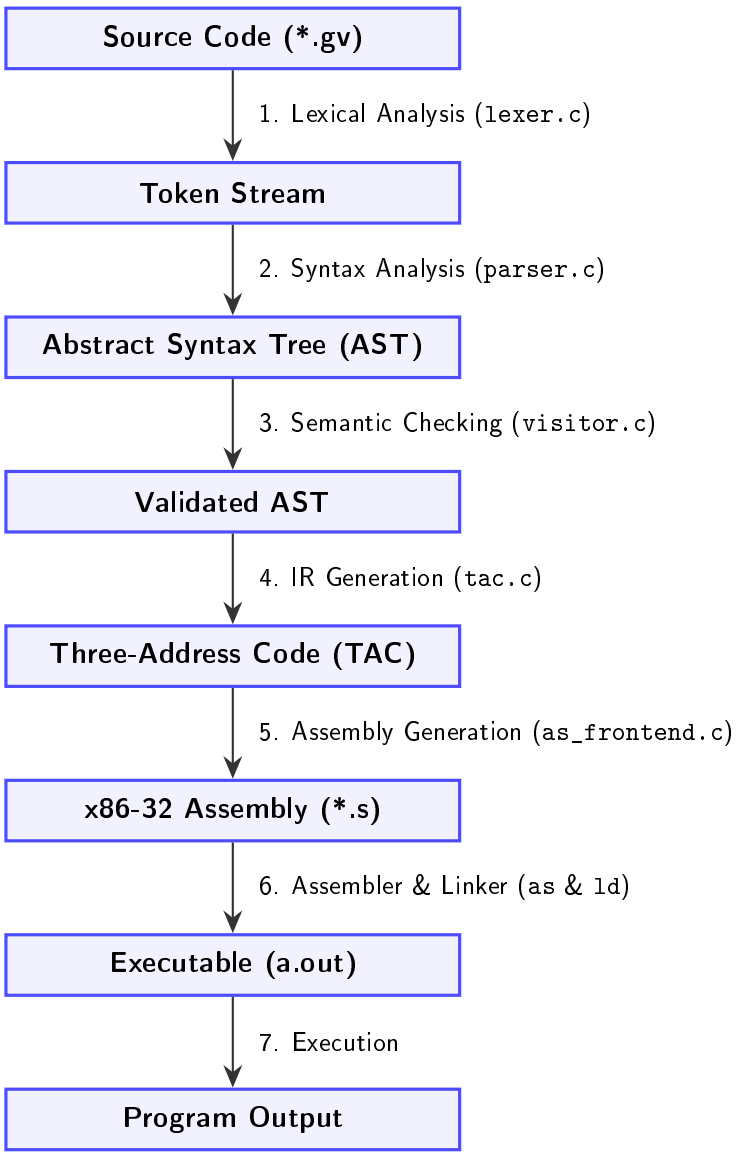
Testing Strategy: Each test file validates specific language features through end-to-end execution and output verification.

File Pattern	Description
`*.txt`	Source code test files. Each file contains a complete program in the GVR language testing specific features: arithmetic operations, control flow (loops, conditionals), functions, arrays, I/O operations, and edge cases.
`*.expected`	Expected output files. Contains the exact output (including whitespace and newlines) that the compiled program should produce when executed. The test runner performs byte-exact comparison against actual output.
`*.input`	Input data files (optional). Contains input data to be provided to the program via stdin during test execution. Required for programs using `input()` or `to_int(input())`. Data is fed line-by-line to the executing program.
`factorial.txt`	Tests factorial calculation using loops and multiplication. Validates loop constructs, arithmetic operations, and variable updates.
`fibonacci.txt`	Tests Fibonacci sequence generation. Validates loops, variable assignments, and arithmetic operations.
`arithmetic.txt`	Tests all arithmetic operators: `+`, `-`, `*`, `/`, `%`, unary `-`. Validates operator precedence and associativity.
`mergesort.txt`	Tests merge sort implementation. Validates arrays, recursion, function calls, and complex control flow.
`conditionals.txt`	Tests if-else statements and boolean expressions. Validates comparison operators and logical operators (`&&`, `||`, `!`).
`loops.txt`	Tests while and for loop constructs. Validates loop initialization, condition checking, and iteration.
`arrays.txt`	Tests array declaration, indexing, multi-dimensional arrays, and array operations.
`functions.txt`	Tests function definitions, calls, parameter passing, return values, and recursion.

3.5 Generated Files (During Compilation)

File	Description
`a.s`	**Generated Assembly:** Contains the compiled x86-32 AT&T assembly code, including the program's data and text sections.
`a.o`	**Assembled Object:** Unlinked binary machine code produced by the GNU assembler (`as -32`).
`a.out`	**Final Executable:** The fully linked, runnable ELF32 binary produced by the GNU linker (`ld -m elf_i386`).
`*.o` (in `src/`)	**Compiler Build Files:** Intermediate object files generated when compiling the compiler itself. Removed by running `make clean`.

3.6 File Interaction Workflow



3.7 Directory Tree Structure

```
GVR-Compiler/
├── Docs/ ..... Documentation
│   └── Documentation.pdf
├── Makefile ..... Build configuration
├── README.md ..... Project overview
├── run_tests.py ..... Test automation script
├── src/ ..... Source code
│   ├── as_frontend.c ..... Assembly generator
│   ├── AST.c ..... AST implementation
│   ├── builtins.c ..... Standard library
│   ├── include/ ..... Header files
│   │   ├── as_frontend.h
│   │   ├── AST.h
│   │   ├── builtins.h
│   │   ├── io.h
│   │   ├── lexer.h
│   │   ├── list.h
│   │   ├── macros.h
│   │   ├── parser.h
│   │   ├── tac.h
│   │   ├── token.h
│   │   ├── types.h
│   │   └── visitor.h
│   ├── io.c ..... File management
│   ├── lexer.c ..... Lexical analyzer
│   ├── list.c ..... Data structures
│   ├── main.c ..... Entry point
│   ├── parser.c ..... Syntax analyzer
│   ├── tac.c ..... Intermediate representation
│   ├── token.c ..... Token definitions
│   ├── types.c ..... Type system
│   └── visitor.c ..... AST traversal
└── tests/ ..... Test suite
    ├── 2d_array.expected
    ├── 2d_array.txt
    ├── check_input.expected
    ├── check_input.input
    ├── check_input.txt
    ├── logic.expected
    ├── logic.txt
    ├── loop_10.expected
    ├── loop_10.txt
    ├── math.expected
    ├── math.txt
    ├── merge_sortstress.expected
    ├── merge_sortstress.txt
    ├── prime.expected
    └── prime.txt
```

4 Language Features

4.1 Data Types

- **Integers:** Signed 32-bit integers
- **Variables:** Named storage locations
- **Arrays:** Fixed-size contiguous memory blocks
- **Multi-dimensional Arrays:** N-dimensional array support

N-Dimensional Array Declaration:

> The compiler supports multi-dimensional arrays with flexible declaration syntax. Arrays are stored in row-major order for efficient memory access.

```
arr1D:int[10]; # 1D Array

matrix:int[5][10];    # 2D Array (5 rows, 10 columns)

cube:int[3][4][5];    # 3D Array

# Example: 2D array initialization and access
main = () -> {
        # Declare a 3x3 matrix
        matrix:int[3][3];

        # Initialize values
        i:int = 0;
        while(i < 3){
                j:int = 0;
                while(j < 3){
                        matrix[i][j] = i * 3 + j;
                        j += 1;
                }
                i += 1;
        }

        # Access and print
        print(matrix[1][2], "\n"); # Output: 5

        return 0;
}
```

Memory Layout for Multi-dimensional Arrays:

```
# For int[3][4]:
# Element [i][j] is at: base_address + (i * 4 + j) * sizeof(int)

# For int[2][3][4]:
# Element [i][j][k] is at:
#     base_address + ((i * 3 + j) * 4 + k) * sizeof(int)
```

4.2 Control Flow

- **Conditional:** `if`, `elif`, `else` statements
- **Loops:** `while`, `for` iterations
- **Functions:** User-defined function calls

4.3 Operations

- **Arithmetic:** `+`, `-`, `*`, `/`, `%`
- **Comparison:** `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical:** `&&`, `||`, `!`
- **Assignment:** `=`, `+=`, `-=`, `*=`, `/=`, `%=`

4.4 Input/Output

```
num:int = to_int(input()); # Read Integer
str:string = input(); # Read string
line:string = input_line(); # Read the whole line

print(num, "\n"); # Similar in functionality to Python's print
```

5 Compiler Architecture

5.1 Compilation Stages

Stage	Description
1. Lexical Analysis	Tokenization of source code into meaningful symbols
2. Syntax Analysis	Parse tree generation and grammar validation
3. Semantic Analysis	Type checking, scope resolution, and validation
4. Intermediate Code	Three-address code (TAC) generation
5. Optimization	Constant folding, dead code elimination
6. Code Generation	x86 assembly code output

5.2 Compile-Time Error Detection

The semantic analyzer performs extensive static analysis to detect errors before code generation, improving reliability and developer experience.

5.2.1 Division by Zero Detection

The semantic analyzer detects division by zero at compile time when the divisor is a constant expression:

```
# Detected at compile time - ERROR
x:int = 10 / 0;        # Error: Division by zero
y:int = 25 % 0;        # Error: Modulo by zero
z:int = (5 + 3) / (2-2); # Error: Division by zero

# Cannot be detected at compile time (runtime check needed)
a:int = to_int(input());
b:int = 10 / a;        # Runtime check required
```

Error Message Example:

```
[Math Error] Line 2: Division by literal zero detected!
```

Implementation Details:

- **Constant Folding:** Evaluates constant expressions during semantic analysis
- **Zero Detection:** Checks divisor value before performing division
- **Expression Analysis:** Recursively analyzes arithmetic expressions
- **Error Recovery:** Reports error but continues compilation to find other issues

Additional Compile-Time Checks:

- Type compatibility in assignments and operations
- Undeclared variable usage
- Array bounds in constant index expressions
- Function signature matching
- Return type consistency

5.3 Register Usage

The compiler follows x86-32 calling conventions:

Register	Purpose
`%eax`	Return values, system call numbers
`%ebx`	System call arguments, base pointer
`%ecx`	Counter register for loops
`%edx`	Data register
`%esp`	Stack pointer
`%ebp`	Base pointer for stack frames

5.4 Core System and Standard Library Functions

Component / Function	Description
`_start`	**Program Initialization:** The true entry point. Allocates a 64MB heap via `mmap2`, sets up the stack, calls `main`, and handles a clean program exit.
`main`	**User Entry Point:** The default execution block for compiled code. Sets up a standard C-style stack frame before executing core logic.
`builtin_print_str`	**String Output:** Prints a string to standard output. Calculates length via the null-terminator (`0`) before invoking `sys_write`.
`builtin_print_int`	**Integer Output:** Converts signed integers to ASCII strings by repeatedly dividing by 10 (handling negatives) and prints the result buffer.
`builtin_print_char`	**Character Output:** Temporarily writes a single byte to the local stack frame and prints it directly to the console.
Memory Management	**Global State:** Allocates a 1KB input buffer and 64MB heap in the `.bss` section, with tracking pointers initialized in `.data`.
`builtin_input`	**Reading User Input:** Reads characters via `sys_read`. Skips leading whitespace and captures input until it hits a space or newline.
`builtin_to_int`	**String-to-Integer:** Acts as a standard `atoi` function. Parses numerical string characters and mathematical signs into a raw, signed integer.

6 Building the Compiler

6.1 Prerequisites

- **GCC/G++:** C/C++ compiler with multilib support
- **Make:** Build automation tool
- **Binutils:** GNU assembler (`as`) and linker (`ld`)
- **Python 3:** For test automation
- **32-bit Libraries:** `gcc-multilib`, `g++-multilib`

6.2 Installation (Ubuntu/Debian)

```
sudo apt-get update
sudo apt-get install build-essential gcc-multilib g++-multilib \
binutils python3
```

6.3 Build Commands

Command	Purpose
`make`	Build the compiler
`make clean`	Remove build artifacts
`make clean && make`	Rebuild from scratch

7 Usage

7.1 Compiling a Program

Basic compilation workflow:

```
./gvr.out program.txt # Generates assembly and executable
```

7.2 Executing

```
./a.out
```

7.3 Complete Workflow

```
# 1. Write source file
# hello.txt

# 2. Compile
./gvr.out hello.txt

# 3. Execute
./a.out
```

8 Automated Testing Framework

The compiler includes a comprehensive automated testing framework that validates generated code through a complete pipeline from compilation to execution verification.

8.1 Test Pipeline

Stage	Description
1. Compile	Execute `./gvr.out source.txt` to generate assembly (`a.s`)
2. Execute	Run executable with optional input (3-second timeout)
3. Validate	Compare actual output with expected output
4. Cleanup	Remove temporary files (`.s`, `.o`, executables)

8.2 Running Tests

Execute the complete test suite:

```
python3 run_tests.py
```

Example output:

```
Starting Compiler Test Suite...

Testing factorial.txt.............. [ PASS ]
Testing fibonacci.txt.............. [ PASS ]
Testing arithmetic.txt............. [ PASS ]

========================================
Test Run Complete: 3 Passed, 0 Failed.
========================================
```

9 Programming Examples

9.1 Example 1: Factorial Calculation

```
main = () -> {
	n:int = to_int(input());
	result:int = 1;
	counter:int = 1;

	while(counter <= n){
		result *= counter;
		counter += 1;
	}

	print(result, "\n");

	return 0;
}
```

9.2 Example 2: Merge Sort Algorithm

```
# Merge two sorted subarrays (void function)
merge:void = (arr:int[], left:int, mid:int, right:int) -> {
	n1:int = mid - left + 1;
	n2:int = right - mid;

	# Create temporary arrays
	L:int[100];
	R:int[100];

	# Copy data to temp arrays
	i:int = 0;
	while(i < n1){
		L[i] = arr[left + i];
		i += 1;
	}
	j:int = 0;
	while(j < n2){
		R[j] = arr[mid + 1 + j];
		j += 1;
	}

	# Merge the temp arrays back
	i = 0;
	j = 0;
```

```
                        k:int = left;
                        while(i < n1 && j < n2){
                                if(L[i] <= R[j]){
                                        arr[k] = L[i];
                                        i += 1;
                                }
                                else{
                                        arr[k] = R[j];
                                        j += 1;
                                }
                                k += 1;
                        }

                        # Copy remaining elements
                        while(i < n1){
                                arr[k] = L[i];
                                i += 1;
                                k += 1;
                        }
                        while(j < n2){
                                arr[k] = R[j];
                                j += 1;
                                k += 1;
                        }
                }

                # Recursive merge sort (void function)
                mergeSort:void = (arr:int[], left:int, right:int) -> {
                        if(left < right){
                                mid:int = left + (right - left) / 2;
                                # Sort first and second halves
                                mergeSort(arr, left, mid);
                                mergeSort(arr, mid + 1, right);
                                # Merge the sorted halves
                                merge(arr, left, mid, right);
                        }
                }

                main = () -> {
                        n:int = to_int(input());
                        arr:int[1000];

                        # Read array elements
                        i:int = 0;
                        while(i < n){
                                arr[i] = to_int(input());
                                i += 1;
                        }
```

```
73
74                      # Sort the array
75                      mergeSort(arr, 0, n - 1);
76
77                      # Print sorted array
78                      i = 0;
79                      while(i < n){
80                              print(arr[i], " ");
81                              i += 1;
82                      }
83                      print("\n");
84
85                      return 0;
86              }
```

10 Performance Metrics

10.1 Execution Time Comparison

The GVR compiler generates efficient x86 assembly code that demonstrates competitive performance compared to other languages. The following benchmark compares execution times for sorting 15,000 randomly generated integers using the merge sort algorithm:

Test	GVR Language	C (gcc -O2)	Python 3
Sorting (15,000 numbers)	0.033s	0.003s	0.107s

Key Insights:

- **GVR vs C:** GVR is approximately 11× slower than optimized C code
- **GVR vs Python:** GVR is approximately 3.2× faster than Python
- Demonstrates benefits of ahead-of-time compilation to native machine code

Future Optimizations:

- Advanced constant propagation
- Loop optimization and unrolling
- Improved register allocation and support for floating-point numbers
- Instruction selection optimization
- Function inlining for small functions

11 Technical Specifications

11.1 Target Platform

- **Architecture:** x86 32-bit (IA-32)
- **Operating System:** Linux
- **Assembly Format:** AT&T syntax
- **Executable Format:** ELF32
- **System Calls:** Linux syscall interface

11.2 System Call Interface

```
                mov eax, 1      ; sys_exit
                mov ebx, 0      ; status code
                int 0x80        ; system call

                mov eax, 4      ; sys_write
                mov ebx, 1      ; stdout
                int 0x80

                mov eax, 3      ; sys_read
                mov ebx, 0      ; stdin
                int 0x80
```

12 Conclusion

The GVR Compiler provides a complete solution for compiling high-level programs to x86 assembly. The integrated testing framework ensures code quality through automated validation of compiler output. By combining robust compilation with comprehensive testing, GVR enables reliable software development and facilitates compiler debugging and enhancement.

The modular architecture, detailed documentation, and extensive test suite make GVR suitable for both educational purposes and practical compiler development projects. The automated testing framework significantly reduces manual testing effort while providing detailed diagnostics for rapid debugging.

With features such as n-dimensional array support, compile-time error detection, and competitive performance characteristics, the GVR compiler demonstrates the effective implementation of modern compiler techniques while maintaining code clarity and educational value.